

100+



# Linux DevOps Interview Questions and Answers



Essential Linux-only interview prep

**Kernel**Modules, Scheduling,  
Syscalls & more**Boot Process**BIOS, GRUB,  
Systemd & more**Permissions**Users, Groups,  
ACLs & more**Processes**Threads, Signals,  
IPC & more**Networking**TCP/IP, DNS,  
Firewall & more**File Systems**Ext4, XFS, Btrfs,  
Mounts & more**Security**SSH, SELinux,  
AppArmor & more**Troubleshooting**Logs, Performance,  
Debugging & more

★ FEATURED

## Linux Interview Prep Guide

Curated Linux-only interview questions  
with detailed answers to help you **crack**  
**your next** DevOps interview.



OPEN GUIDE

Compiled by **Pankaj K.**



PDF EDITION / CURATED PRACTICE SET

# 100+ Linux DevOps Interview Questions & Answers

A redesigned interview guide built to match the neon terminal cover style. Use it to review Linux fundamentals, diagnose production scenarios, and prepare for DevOps conversations with confidence.



121

QUESTIONS

8

CORE DOMAINS

100%

LINUX FOCUS

**KERNEL**

Modules, scheduling, syscalls



**BOOT**

BIOS, GRUB, systemd



**PERMISSIONS**

Users, groups, ACLs



**PROCESSES**

Threads, signals, IPC



**NETWORKING**

TCP/IP, DNS, firewall



**FILESYSTEMS**

Ext4, XFS, Btrfs



**SECURITY**

SSH, SELinux, AppArmor



**TROUBLESHOOT**

Logs, tracing, performance



**Built for fast revision:** Scan the tracks, review the Q&A cards, and revisit the advanced diagnostics before interviews.



NAVIGATION / READING MAP

# Guide Roadmap

Six focused tracks across 121 Linux DevOps interview prompts.

01

**Linux Foundations & Core Internals**  
Kernel, user space, boot flow, permissions, process basics, monitoring, and /proc.

Q001-Q019

02

**Administration, Storage & Access Control**  
PIDs, search, RAID, mounts, SSH, package management, cron, automation, and users.

Q020-Q041

03

**Troubleshooting, Logs & Runtime Diagnostics**  
Signals, boot logs, resource bottlenecks, I/O, kernel panic, backups, and configuration.

Q042-Q058

04

**Linux for DevOps, Cloud & Reliability**  
IaC, containers, Kubernetes, CI/CD, secrets, scaling, deployments, observability, and...

Q059-Q073

05

**Advanced Systems Management**  
Concurrency, cgroups, log systems, ulimit, fstab, NFS, LVM, modules, sudo, and servi...

Q074-Q096

06

**Deep-Dive Performance, Kernel & Security**  
Memory, cgroups v2, DNS caching, syscalls, live patching, DNSSEC, SMART, strace, p...

Q097-Q121



TRACK 01 / Q001-Q019

# Linux Foundations & Core Internals

**01**  
TRACK

Kernel, user space, boot flow, permissions, process basics, monitoring, and /proc.

Q001



## What is the Linux kernel?

The Linux kernel is the core component of the operating system that interacts directly with hardware. It manages CPU scheduling, memory, devices and processes, providing a hardware abstraction layer. Because it is released under the GNU General Public License, the source code can be viewed, modified and redistributed by anyone.

Q002



## Explain the difference between kernel space and user space.

Kernel space is a protected area of memory where the kernel runs with full privileges (ring 0). User space is where user applications run with restricted privileges. Separating these spaces prevents user processes from directly accessing kernel memory, improving stability and security.

Q003



## Describe the Linux boot process.

When a Linux system boots, the BIOS/UEFI performs hardware initialization and loads a bootloader such as GRUB. The bootloader loads the kernel into memory and hands off control. The kernel initializes hardware, mounts the root filesystem and starts the initial process ('systemd' or 'init'). This process spawns other services and eventually presents a login prompt.

Q004



## What happens when you run the ls command in a shell?

The shell parses the command and searches for the ls executable in the directories listed in \$PATH. It then uses the fork() system call to create a child process and execve() to replace the child's memory with the ls program. The program reads directory entries from the underlying filesystem and prints them to standard output. When finished, it exits and returns control to the shell.

Q005



## What is the purpose of the /etc directory?

/etc contains system-wide configuration files and subdirectories. It includes user account information (passwd), host names (hosts), system services configuration, and application-specific settings. Modifying files in /etc typically requires administrative privileges.



Q006

**How do processes differ from threads?**

A process has its own memory space and resources. Creating a process is relatively expensive and processes are isolated from each other. A thread is a lighter-weight unit of execution that shares the process's memory space. Threads have lower creation overhead but must coordinate access to shared data.

Q007

**What are file permissions in Linux and how are they represented?**

Each file has permissions for the owner (user), group and others. The basic permissions are read (r), write (w) and execute (x). Permissions can be displayed symbolically (e.g., `rw-r--r--`) or numerically (octal), where read=4, write=2, execute=1; e.g., 755 means `rx` for the owner, `rx` for the group and `-` for others.

Q008

**How can you change file permissions and ownership?**

Use `chmod` to change permissions, `chown` to change the owner and `chgrp` to change the group. For example, `chmod 755 script.sh` sets the permission to `rw-r--r--`, and `chown alice:developers file` changes the owner to `alice` and group to `developers`.

Q009

**What is a hard link versus a soft (symbolic) link?**

A hard link is another directory entry pointing to the same inode as the original file; it must reside on the same filesystem and remains valid even if the original file is deleted. A soft link (symbolic link) is a special file that stores a path to the target; it can span filesystems but becomes dangling if the target is moved or deleted.

Q010

**How do you list currently running processes?**

Commands such as `ps`, `top` and `htop` display running processes. `ps aux` lists all processes; `top` shows a dynamic view of CPU, memory and process information; `htop` provides a more interactive interface with sorting and filtering.

Q011

**How do you check disk usage on a Linux system?**

Use `df` to display filesystem usage; `df -h` shows human-readable sizes. Use `du` to estimate directory or file usage (e.g., `du -sh /var/log`). `ncdu` is an interactive tool for navigating disk usage.

Q012

**What is swap space and why is it used?**

Swap space is disk space used as an extension of RAM. When physical memory is exhausted, the kernel can page inactive memory pages to swap, freeing RAM. Swap can be a dedicated partition or a file. It is much slower than RAM, so adequate physical memory is important for performance.





Q013

**Explain the difference between /etc/profile, .bashrc and .bash profile.**

/etc/profile is a global shell initialization script executed for login shells. .bash profile (or .profile) is executed for a user's login shell and can set environment variables. .bashrc is executed for non-login interactive Bash shells and typically contains command aliases and functions.

Q014

**How do you find which process is listening on a particular port?**

Use `ss -tulpn` or `netstat -tulpn` to list listening ports and associated processes. The `lsof -i :PORT` command shows the specific process using a given port.

Q015

**What is load average, and how do you interpret it?**

Load average represents the average number of running and waiting processes over one, five and fifteen minutes. For a single-CPU system, a load average of 1 means the CPU is fully utilized. On multi-core systems, divide the load average by the number of cores to interpret CPU load.

Q016

**What is the purpose of nice and renice?**

`nice` starts a process with a specified scheduling priority (niceness). A higher niceness value reduces a process's priority, allowing other processes to use more CPU. `renice` changes the niceness of an existing process, useful for throttling resource-intensive processes.

Q017

**How do you monitor system performance on Linux?**

Use `top`, `htop` and `uptime` for CPU and load information. `vmstat`, `iostat` and `pidstat` show memory, disk I/O and per-process statistics. `free -m` reports memory usage. Tools such as `sar` (from `sysstat`) provide historical metrics.

Q018

**How do you check network connectivity and diagnose DNS issues?**

Commands like `ping` verify connectivity to a host. `traceroute` shows the network path. For DNS resolution, use `nslookup` or `dig` to query DNS records. `host` provides a simple forward and reverse lookup.

Q019

**What is the purpose of the /proc filesystem?**

/proc is a pseudo-filesystem that exposes kernel and process information as files. Entries like `/proc/cpuinfo` and `/proc/meminfo` show hardware details, while `/proc/PID` directories contain files representing each process's state, memory and file descriptors.



TRACK 02 / Q020-Q041

# Administration, Storage & Access Control

**02**  
TRACK

PIDs, search, RAID, mounts, SSH, package management, cron, automation, and users.

Q020



## How can you find the process ID (PID) of a running process?

Use `ps` or `pgrep`. For example, `pgrep sshd` returns the PID of the `sshd` process. Alternatively, `ps -ef | grep process name` lists the process with its PID.

Q021



## What command would you use to search for files by name, size or type?

The `find` command searches directories recursively. For example, `find /var/log -name '*.log'` finds log files; `find /home -type f -size +100M` finds files over 100 MB; `find /tmp -type d -perm 700` finds directories with specific permissions.

Q022



## Describe RAID and its common levels.

RAID (Redundant Array of Independent Disks) combines multiple drives for performance and/or redundancy. RAID 0 stripes data across disks for speed but no redundancy; RAID 1 mirrors data across disks for fault tolerance; RAID 5 and RAID 6 use striping with distributed parity for a balance of performance and redundancy; RAID 10 combines mirroring and striping.

Q023



## How do you mount and unmount filesystems?

Use `mount` to attach a device to the filesystem hierarchy: `sudo mount /dev/sdb1 /mnt/data`. To unmount, ensure the mount point is not in use and run `sudo umount /mnt/data`. The `lsblk` command helps identify devices and partitions.

Q024



## How do you secure a Linux server?

Apply security updates and patches promptly, disable unused services, configure a firewall (`iptables`, `nftables` or `firewalld`), enforce strong authentication (SSH keys), limit root login, use tools such as SELinux or AppArmor for mandatory access control, set appropriate file permissions, and regularly review logs. Implement intrusion detection and maintain reliable backups.

Q025



## What are cron jobs and how do they differ from systemd timers?

Cron is a scheduling service that runs commands at specified times based on entries in crontab files. Systemd timers are units managed by `systemd` that can trigger services on a schedule or after events. Timers integrate with `systemd`'s dependency management and logging and are often used instead of cron for newer systems.



Q026

**How do you check and modify scheduled cron jobs?**

Use `crontab -l` to list the current user's cron jobs and `crontab -e` to edit them. System-wide cron jobs are stored in `/etc/crontab` and the `/etc/cron.` directories. For systemd timers, use `systemctl list-timers` and edit the timer and service unit files.

Q027

**What is SELinux (or AppArmor) and how do you check its status?**

SELinux and AppArmor are mandatory access control (MAC) systems that confine processes to least privilege. To check SELinux status on Red Hat-based systems, use `sestatus` or `getenforce`. AppArmor status can be checked with `aa-status`. Policies can be set to enforcing, permissive or disabled modes.

Q028

**How do you manage software packages?**

Different distributions use different package managers: Debian/Ubuntu use `apt` and `dpkg`; Red Hat/CentOS use `yum` (or `dnf`) and `rpm`; SUSE uses `zypper`. Packages can be installed, upgraded or removed via these tools (e.g., `sudo apt update && sudo apt install git`).

Q029

**How do you set up passwordless SSH between two hosts?**

Generate an SSH key pair with `ssh-keygen` on the client, then copy the public key (`id_rsa.pub`) to the server's `~/.ssh/authorized_keys` using `ssh-copy-id user@server`. Ensure correct permissions on the `.ssh` directory. The client can then connect without entering a password.

Q030

**How do you automate repetitive tasks on Linux?**

Shell scripts (Bash) are commonly used for automation, as are higher-level languages like Python. Scripts can combine commands, loops and conditionals to perform complex tasks. Configuration management tools such as Ansible, Puppet or Chef automate provisioning and configuration across multiple machines.

Q031

**How do you configure and manage a firewall on Linux?**

Use `iptables` or `nftables` to define low-level packet filtering rules. Higher-level tools like `firewalld` provide zones and services for easier management (e.g., `sudo firewall-cmd --permanent --add-service=https`). Always test rules to avoid locking yourself out of remote access.

Q032

**What is systemd and how does it differ from SysV init?**

`systemd` is the modern init system used by most Linux distributions. It starts services in parallel, manages dependencies and provides features such as sockets, timers and cgroups. `SysV init` is the older, sequential init system. `systemd` offers better boot performance and integrated management tools (`systemctl`).





Q033

**How would you troubleshoot a service that failed to start?**

Use `systemctl status service name` to view status and recent logs. Check `journalctl -u service name` for detailed logs. Verify configuration files, permissions and dependencies. If using SysV init, check `/var/log/messages` and the service's init script.

Q034

**How do you check which process consumes high CPU or memory?**

Run `top` or `htop` to view CPU and memory usage per process. Use `ps -eo pid,comm,%cpu,%mem --sort=-%cpu | head` for a quick overview. `vmstat` and `pidstat` provide more detailed statistics.

Q035

**How do you find which process is listening on a particular port?**

Use `sudo lsof -i :PORT` or `sudo ss -ltnp`. For example, `sudo ss -ltnp | grep 22` shows the process listening on port 22.

Q036

**How do you analyze logs in Linux for troubleshooting?**

System logs are typically stored under `/var/log`. Use `journalctl` on systemd systems to query logs. Tools like `grep`, `awk` and `less` help search and view logs. For web servers, check `/var/log/httpd` or `/var/log/nginx`. Centralized logging solutions like `rsyslog` or `ELK` stack can aggregate logs across servers.

Q037

**What is the `/proc` filesystem used for?**

`/proc` is a virtual filesystem that provides process and kernel information. It contains directories named after process IDs, which include details such as environment variables (`cmdline`), open file descriptors (`fd`), memory maps (`maps`) and status (`status`). It also contains system-wide files like `sys/fs/file-nr`.

Q038

**How do you list all open files in Linux?**

Use `lsof` to list open files and the processes that opened them. For example, `lsof -p PID` lists files opened by a specific process. To find which process has a file open, use `lsof filename`.

Q039

**What is an inode and how can you check inode usage?**

An inode is a data structure containing metadata about a file (permissions, timestamps, pointers to data blocks). Use `df -i` to view inode usage on each filesystem. Running out of inodes can prevent file creation even when space is available.

Q040

**How do you create a user and add them to a group?**

Use `sudo useradd username` to add a user. Use `passwd username` to set a password. To add the user to a group, run `sudo usermod -aG groupname username`. Alternatively, create the user and set the primary group at once: `sudo useradd -g groupname username`.



Q041



How do you check scheduled cron jobs for all users?

System-wide cron jobs are stored in /etc/crontab and in the /etc/cron.d, /etc/cron.daily, /etc/cron.hourly directories. To see a specific user's cron jobs, use `sudo crontab -u username -l`. For systemd timers, use `systemctl list-timers`.



TRACK 03 / Q042-Q058

# Troubleshooting, Logs & Runtime Diagnostics

**03**  
TRACK

Signals, boot logs, resource bottlenecks, I/O, kernel panic, backups, and configuration.

Q042



## What is the difference between kill, pkill and killall?

kill sends a signal to a specific process ID (PID). pkill sends a signal to processes matching a name or other attributes. killall sends a signal to all processes with a given name. All three default to sending SIGTERM but can send other signals (e.g., SIGKILL).

Q043



## How do you view system boot logs?

On systems using systemd, run journalctl -b to view logs from the last boot. Alternatively, check /var/log/boot.log or /var/log/dmesg. The dmesg command prints kernel ring buffer messages.

Q044



## What are the differences between top, htop and atop?

top shows real-time process statistics in a non-graphical interface. htop provides a more interactive, color-coded display with support for sorting and filtering. atop collects system and process statistics over time, useful for analyzing resource usage historically.

Q045



## What tools can you use to monitor system performance?

Use top/htop for CPU and memory, vmstat for virtual memory, iostat for disk I/O, pidstat for per-process statistics and sar for aggregated metrics. More advanced tools include perf, bcc-tools and eBPF observability frameworks.

Q046



## How do you troubleshoot a slow Linux server?

Check CPU, memory, disk and network usage. Use top/htop for CPU/memory, iostat for disk I/O, vmstat for swapping and sar for historical trends. Examine system logs under /var/log for errors, and ensure no runaway processes or resource leaks exist.

Q047



## What is a kernel panic, and how do you troubleshoot it?

A kernel panic occurs when the kernel encounters a fatal error and cannot continue. Troubleshoot by examining error messages on the console and in logs (/var/log/kern.log, /var/log/messages). Boot into rescue mode, check hardware (RAM, disk), update firmware and kernels, and apply vendor updates or patches.



Q048

**How do you troubleshoot high disk I/O?**

Use `iostat -xz 1` or `iotop` to see which processes generate I/O. Check disk health with SMART tools. Consider moving I/O-intensive processes to separate disks or using RAID. Evaluate filesystem choices and mount options (e.g., `noatime`). For network storage, inspect network throughput and latency.

Q049

**How do you handle a "filesystem is full" error when `df` shows free space?**

Check inode usage with `df -i`. If inodes are exhausted, delete unnecessary small files. Use `ls -l +L1` to find deleted files still held open by processes; restarting those processes frees the space.

Q050

**Explain the difference between stack and heap memory.**

Stack memory stores function call frames and local variables. It grows and shrinks automatically as functions are called and return. Heap memory is dynamically allocated using functions like `malloc()` and must be manually freed. Heap allocations persist beyond function scope and can lead to memory leaks if not freed.

Q051

**What is a memory leak?**

A memory leak occurs when a program fails to release memory that is no longer needed. Leaked memory cannot be reused by the program or the system, eventually exhausting available memory. Detect leaks with tools such as `valgrind`, `leak-sanitizer` and by monitoring RSS over time.

Q052

**How does Linux handle interrupts?**

Hardware devices send interrupts to notify the CPU of events. The CPU temporarily suspends its current task and executes an interrupt handler in kernel space. Handlers are registered for each interrupt and perform minimal work, often deferring longer processing to bottom-half mechanisms such as `softirqs` or `tasklets`.

Q053

**What is load average and why is it important?**

Load average reflects the average number of runnable or waiting processes. It helps assess system utilization and whether the CPU is a bottleneck. Sustained high load relative to CPU cores indicates CPU overutilization; low load with poor performance may point to I/O or memory constraints.

Q054

**What happens when you run `curl example.com`?**

`curl` resolves the domain via DNS, establishes a TCP connection (3-way handshake) to the server's IP and port, sends an HTTP request and receives the response. It then prints the response or saves it to a file. Options allow following redirects, setting headers, performing POST requests and more.



Q055

**How do you handle log rotation on Linux?**

The logrotate utility automatically rotates, compresses and removes old log files according to configuration files in `/etc/logrotate.d`. Systemd's journald stores logs in binary format and manages its own rotation settings via `/etc/systemd/journald.conf`.

Q056

**What is a daemon, and how do you manage daemons?**

A daemon is a background process that provides a service (e.g., `sshd`, `crond`). Daemons typically have names ending in `d`. On systems using `systemd`, manage daemons with `systemctl start|stop|restart service`. Use `systemctl enable service` to start on boot.

Q057

**How do you create backups of a directory on Linux?**

Use `tar -czvf backup.tar.gz /path/to/dir` to create a compressed archive. `rsync -av --delete /source/ /destination/` performs incremental backups. Consider using the 3-2-1 rule (three copies, two types of media, one off-site) for reliable backups.

Q058

**How do you automate system configuration across multiple servers?**

Configuration management tools such as Ansible, Puppet, Chef and Salt allow you to define system configuration as code. They can install packages, manage files and services, enforce security settings and integrate with CI/CD pipelines.





TRACK 04 / Q059-Q073

# Linux for DevOps, Cloud & Reliability

**04**  
TRACK

IaC, containers, Kubernetes, CI/CD, secrets, scaling, deployments, observability, and HA.

Q059



## What is Infrastructure as Code (IaC)?

IaC is the practice of managing and provisioning infrastructure (servers, networks, storage) through machine-readable definition files instead of manual processes. Tools like Terraform, CloudFormation and Pulumi describe infrastructure in code that can be version-controlled, peer-reviewed and automatically applied.

Q060



## How do containers differ from virtual machines?

Containers share the host OS kernel but isolate processes using namespaces and control groups. Virtual machines emulate hardware and run separate kernels. Containers are lightweight, start quickly and use fewer resources. VMs offer stronger isolation at the cost of overhead.

Q061



## How do you deploy and manage Docker containers on Linux?

Install Docker, build images using Dockerfiles, run containers with `docker run`, manage containers with `docker ps` and `docker stop/start`. Use `docker-compose` to define multi-container applications. For orchestration across clusters, use Kubernetes or Docker Swarm.

Q062



## What is Kubernetes, and why is it used?

Kubernetes is an open-source container orchestration platform that automates deployment, scaling and management of containerized applications. It handles scheduling, load balancing, service discovery, rolling updates and self-healing, making it ideal for microservices architectures.

Q063



## How do you implement CI/CD pipelines for Linux applications?

Use tools like Jenkins, GitLab CI, GitHub Actions or CircleCI. A typical pipeline includes steps for code checkout, compilation or building containers, running tests, performing security scans and deploying to staging or production. Pipelines can run shell commands and interact with cloud services or configuration management tools.

Q064



## What is immutable infrastructure?

Immutable infrastructure is a pattern where servers are not modified after deployment. Instead, any change is applied by creating a new server image and replacing the old instances. Tools like Packer build machine images, and configuration is applied at build time. Immutable infrastructure reduces configuration drift and improves reproducibility.



Q065

**How does cloud-init help in provisioning cloud instances?**

cloud-init is a set of scripts and utilities used to configure cloud instances at first boot. It can set hostnames, users, SSH keys, packages, and run custom commands. Cloud providers pass user data to cloud-init to automate instance configuration.

Q066

**How do you manage secrets and configuration values securely?**

Store secrets in a dedicated secrets manager (e.g., HashiCorp Vault, AWS Secrets Manager, or Kubernetes Secrets). Do not hard-code secrets in code repositories. Use environment variables or mounted files with restricted permissions. Rotate credentials regularly and restrict access based on least privilege.

Q067

**How do you set up auto-scaling on cloud platforms for Linux workloads?**

Use the cloud provider's auto-scaling service (AWS Auto Scaling, Google Cloud Instance Groups, Azure Scale Sets). Define scaling policies based on metrics such as CPU utilization or custom CloudWatch metrics. Combine with load balancers to distribute traffic across instances.

Q068

**What are rolling updates and blue-green deployments?**

Rolling updates replace old instances with new ones gradually, ensuring minimal downtime. Blue-green deployments run two environments (blue and green); traffic is switched from blue to green once the new version is verified. This allows instant rollback if issues arise.

Q069

**What is observability, and why is it important?**

Observability means having comprehensive insight into a system's internal state through logs, metrics and traces. It allows engineers to understand and diagnose issues, ensuring reliability and performance. Tools like Prometheus, Grafana, Jaeger and ELK stack provide observability across distributed systems.

Q070

**How do you design a highly available and fault-tolerant system using Linux and open-source tools?**

Use load balancers (e.g., HAProxy or NGINX) to distribute traffic across multiple servers. Employ clustering software (Pacemaker/Corosync) to manage failover. Use redundant storage (RAID, Ceph, GlusterFS), replicated databases and regular backups. Implement health checks and monitoring to detect failures and automated recovery.

Q071

**How would you plan capacity for a rapidly growing microservices architecture?**

Analyze current resource usage and growth trends, forecast future demands, identify bottlenecks and plan for headroom. Use auto-scaling to handle unpredictable load, implement rate limiting, and coordinate with development and business teams. Monitor metrics with Prometheus or similar tools and iterate planning as usage patterns change.



Q072



**Describe how you would debug a complex production issue.**

Start by gathering information from logs, metrics and alerts. Try to reproduce the issue in a staging environment. Use a systematic approach such as binary search to narrow down the root cause. Collaborate with other teams and document findings. Once a fix is implemented, perform post-mortem analysis to prevent similar issues.

Q073



**How do you optimize Linux system performance?**

Regularly update the system, tune kernel parameters (e.g., via /etc/sysctl.conf), optimize disk I/O (use appropriate filesystems and mount options), manage memory and swap, disable unnecessary services, and use performance analysis tools to locate bottlenecks. Adjust application settings (e.g., web server worker processes) based on load tests.



TRACK 05 / Q074-Q096

# Advanced Systems Management

**05**  
TRACK

Concurrency, cgroups, log systems, ulimit, fstab, NFS, LVM, modules, sudo, and service envs.

Q074



## What is the difference between a process in running, sleeping and zombie states?

Running processes are executing on the CPU; sleeping processes are waiting for an event (I/O, timer); zombie processes have completed execution but still have an entry in the process table because the parent has not collected the exit status. Zombies consume no resources but should be reaped to avoid table overflow.

Q075



## What is a race condition, and how does Linux handle concurrency?

A race condition occurs when multiple threads or processes access shared data concurrently and the result depends on the order of execution. Linux provides synchronization primitives-mutexes, spinlocks, semaphores and atomic operations-to manage concurrency. Properly locking shared resources prevents race conditions.

Q076



## How do you diagnose memory leaks?

Monitor the resident set size (RSS) of processes over time to detect growth. Use tools like valgrind, massif or heaptrack. For languages with garbage collection, enable profilers or leak detectors (e.g., Python's tracemalloc). Identify objects that persist unexpectedly and free or dereference them appropriately.

Q077



## What commands help troubleshoot network performance issues?

Use ping and mtr to measure latency and packet loss. traceroute displays the route packets take. iperf measures throughput. tcpdump or wireshark capture packets for analysis. Combine these with system metrics to isolate network bottlenecks.

Q078



## How do you inspect and adjust kernel parameters?

Kernel parameters are accessible through the /proc/sys filesystem. Use sysctl -a to list parameters and sysctl -w parameter=value to change them at runtime. Persist changes by adding them to /etc/sysctl.conf or a file under /etc/sysctl.d/.



Q079

**What is the purpose of cgroups and namespaces?**

Control groups (cgroups) limit and account for resource usage (CPU, memory, I/O) for a set of processes. Namespaces isolate resources such as PID numbers, network interfaces and mount points. Together, they form the foundation for containerization, enabling multiple isolated applications to run on the same kernel.

Q080

**How do you manage system logs on Linux?**

Traditional syslog daemons (rsyslog, syslog-ng) handle log routing and filtering. journald stores logs in binary format and exposes them via journalctl. Use logrotate to rotate and compress log files. For centralized logging, forward logs to ELK stack, Graylog or similar systems.

Q081

**What is the difference between soft and hard limits in ulimit?**

Soft limits are the current resource limits for a session and can be increased up to the hard limit. Hard limits are the maximum allowed values set by the system; only root can raise them. Use ulimit -a to view limits and edit /etc/security/limits.conf to set persistent values.

Q082

**How do you set environment variables persistently for a user?**

Add export VAR=value lines to the user's .bashrc or .bash profile. For system-wide environment variables, create scripts in /etc/profile.d/ or edit /etc/environment. Reload the shell or source the file to apply changes.

Q083

**How do you schedule one-off tasks on Linux?**

Use the at command to schedule a job at a specific time (e.g., echo "backup.sh" | at 2am). For modern systems, you can also use systemd-run --on-calendar to run a transient service at a scheduled time.

Q084

**How do you interpret the output of df and du?**

df shows used and available space for mounted filesystems, while du reports disk usage for directories and files. A discrepancy between df and du can occur due to deleted files still open by processes; locate them with lsof +L1.

Q085

**What is the function of the fstab file?**

/etc/fstab defines filesystems to mount at boot. Each line specifies the device, mount point, filesystem type, mount options and dump/pass values. Use sudo mount -a to test fstab entries without rebooting.



Q086

**How do you configure a NFS client and server on Linux?**

Install NFS utilities, export directories on the server via `/etc/exports` and start the NFS server service. On the client, mount the remote export with `mount server:/path /mnt`. Use `nfs4` if available for improved performance and security.

Q087

**How do you monitor and manage RAID arrays?**

For software RAID using `mdadm`, use `mdadm --detail /dev/md0` to view status, and `mdadm --monitor --scan` to receive alerts. Use `lsblk` and `cat /proc/mdstat` to see RAID devices and their state. Replace failed drives and rebuild arrays as necessary.

Q088

**How do you manage LVM (Logical Volume Manager)?**

LVM abstracts storage into physical volumes (PVs), volume groups (VGs) and logical volumes (LVs). Use `pvcreate`, `vgcreate`, `lvcreate` to create storage. Resize LVs with `lvextend`, `lvreduce` and filesystems with `resize2fs` or `xfs growfs`. LVM snapshots provide point-in-time copies.

Q089

**What is a kernel module, and how do you load or unload modules?**

Kernel modules extend kernel functionality (e.g., drivers). Use `lsmod` to list loaded modules, `modprobe module` or `insmod file.ko` to load a module and `modprobe -r module` or `rmmod module` to remove it. Modules can be configured to load at boot via files in `/etc/modules-load.d/`.

Q090

**What is the difference between `apt-get upgrade` and `apt-get dist-upgrade`?**

`apt-get upgrade` installs newer versions of packages without removing or installing dependencies. `apt-get dist-upgrade` may remove or install packages to satisfy dependencies, enabling upgrades that require package changes. Modern systems use `apt full-upgrade` as an alias for `dist-upgrade`.

Q091

**How do you manage services that require high availability?**

Use clustering software (Pacemaker/Corosync) to monitor service health and manage failover. Deploy services behind load balancers and use redundancy at every layer (multiple nodes, multiple data centers). Automate configuration with IaC and monitor with alerting tools to detect failures.

Q092

**How do you compile and install software from source?**

Download the source code, extract it, and read the README or INSTALL. Commonly, run `./configure`, `make` and `sudo make install`. For modern projects using CMake or Meson, follow the build system instructions. Consider using `checkinstall` to create a package and manage uninstallation.



Q093

**What is the function of sudo, and why is it recommended over logging in as root?**

sudo allows permitted users to execute commands with elevated privileges without logging in as root. Using sudo provides an audit trail (via logs), reduces the time spent in an unrestricted account and minimizes the risk of accidental system damage.

Q094

**How do you manage environment variables for system services?**

For systemd services, environment variables can be set in the service unit file using `Environment=` or by specifying an `EnvironmentFile=`. After editing the unit file, reload systemd with `systemctl daemon-reload` and restart the service.

Q095

**What is the purpose of tmux or screen?**

tmux and screen are terminal multiplexers that allow you to manage multiple shell sessions within a single terminal window, detach and reattach to sessions, and keep commands running after disconnecting from SSH. They are valuable for long-running tasks or remote administration.

Q096

**How do you check the current run level or default target?**

On systemd systems, the default target replaces run levels. Use `systemctl get-default` to see the default target (e.g., `multi-user.target`). Traditional SysV systems use `runlevel` or inspect `/etc/inittab`.





TRACK 06 / Q097-Q121

# Deep-Dive Performance, Kernel & Security

06  
TRACK

Memory, cgroups v2, DNS caching, syscalls, live patching, DNSSEC, SMART, strace, perf, and SSH hardening.

Q097



## What is the difference between ps options -e, -f and -u?

-e shows all processes; -f displays full-format listing, including UID, PID, PPID, start time and command; -u selects processes for a user. Combined, ps -ef lists all processes in full format, while ps -u user shows a specific user's processes.

Q098



## How do you interpret the output of free -m?

free -m shows memory usage in megabytes: total, used, free, shared, buffers/cache and available memory. The available column indicates memory available for new processes without swapping. High usage of buffers/cache is normal because the kernel uses spare memory for disk caching.

Q099



## How do you manage kernel upgrades?

Distributions provide packaged kernels via their package managers. Before upgrading, ensure compatibility with hardware and modules. After installation, update the bootloader (usually automatic). Keep at least one previous kernel entry in GRUB to fall back if the new kernel fails to boot.

Q100



## What is cgroups v2, and how does it differ from v1?

Cgroups v2 provides a unified hierarchy for resource control, improved accounting and simplified interface compared to v1's multiple controllers. It solves issues like "transient tasks" and provides better delegation semantics. Modern container runtimes (e.g., systemd and Kubernetes) increasingly use cgroups v2.

Q101



## How do you view hardware information on Linux?

Use lshw, lsblk and lsusb to view hardware details. lspci lists PCI devices; lsusb lists USB devices. dmidecode displays BIOS and motherboard information. dmesg | grep -i usb can show kernel detection of devices.

Q102



## How do you check the status of a RAID array built with mdadm?

Use cat /proc/mdstat to view RAID status and sync progress. mdadm --detail /dev/md0 shows detailed information about an array. Monitoring scripts or mdadm --monitor --scan --daemonise can send email notifications when a drive fails.



Q103

**How do you investigate DNS caching issues?**

Clear the local DNS cache (e.g., `sudo systemd-resolve --flush-caches` on `systemd-resolved`). Query authoritative servers with `dig +trace domain`. Inspect `/etc/resolv.conf` to ensure correct DNS servers. Check if intermediate caching resolvers (e.g., `DNsmasq`) have stale entries.

Q104

**What is the difference between the `fork()` and `exec()` system calls?**

`fork()` creates a new process by duplicating the calling process, resulting in a parent and child. The child receives a copy of the parent's memory space. `exec()` replaces the current process image with a new program. In combination, `fork()` followed by `exec()` spawns new processes.

Q105

**How do you manage kernel parameters for networking performance (e.g., TCP)?**

Tuning parameters in `/proc/sys/net` can improve network performance. Adjust TCP buffers (e.g., `net.ipv4.tcp_rmem`, `tcp_wmem`) and backlog queue length (`net.core.somaxconn`). Apply changes via `sysctl`. Always test changes under expected workloads before deploying widely.

Q106

**How do you check for file descriptor leaks?**

Monitor the number of open file descriptors with `lsdf -p PID | wc -l`. Compare this over time to detect growth. Check the system limits with `ulimit -n` and adjust if necessary. In code, ensure files, sockets and pipes are closed properly.

Q107

**What is a zombie process, and how do you clear zombie processes?**

A zombie process is one that has finished execution but still has an entry in the process table because its parent hasn't read its exit status. Zombies do not use CPU or memory but consume a PID. To clear zombies, ensure the parent process calls `wait()` or terminates; the `init` system will adopt and reap orphaned zombies.

Q108

**How do you monitor and adjust swap usage?**

Use `swapon -s` to list swap devices and `free -m` to view swap usage. The `swappiness` parameter (`/proc/sys/vm/swappiness`) controls the kernel's tendency to use swap. Lower values reduce swap usage. Increase swap space by creating a swap file or partition and enabling it with `mkswap` and `swapon`.

Q109

**What is DNS round robin, and how can it be used for load balancing?**

DNS round robin associates multiple IP addresses with a single DNS record. Each DNS query returns the list in different orders, distributing client connections across servers. While simple, it lacks health checks; combining it with load balancers or health-aware DNS services improves reliability.



Q110



### How do you perform a live kernel patch?

Live kernel patching allows applying security updates without rebooting. Tools like kpatch, ksplice and kgraft load patches as kernel modules. Distribution-specific services such as Canonical Livepatch or Ksplice for Oracle Linux provide commercial support.

Q111



### What is Btrfs, and what advantages does it have over ext4?

Btrfs is a modern copy-on-write (COW) filesystem that supports snapshots, checksums for data and metadata, built-in RAID, and online resizing. Compared to ext4, Btrfs offers advanced features for backups and storage management. However, ext4 remains more mature and stable for some workloads.

Q112



### How do you troubleshoot SELinux denials?

Check audit logs using `ausearch -m avc -ts recent` or `sealert -a /var/log/audit/audit.log`. Use `grep AVC /var/log/audit/audit.log`. If an action is blocked, identify the context and apply the correct SELinux type or boolean, or create a custom policy using `audit2allow`.

Q113



### How do you check disk health and SMART status?

Use `smartctl -a /dev/sdX` (from `smartmontools`) to view SMART attributes, error logs and self-test results. Schedule periodic SMART tests (`smartctl -t short /dev/sdX`). Use `badblocks` to scan for bad sectors.

Q114



### What is DNSSEC, and how can you verify DNSSEC records?

DNSSEC adds cryptographic signatures to DNS records, ensuring authenticity. To verify DNSSEC, use `dig +dnssec example.com` and examine the RRSIG records. Validating resolvers automatically check these signatures; misconfigured DNSSEC can break resolution, so proper key management is essential.

Q115



### How do you configure an NTP or Chrony time service?

Install `ntp` or `chrony`. Edit `/etc/ntp.conf` or `/etc/chrony/chrony.conf` to specify upstream servers (e.g., `pool.ntp.org`). Start and enable the service using `systemctl`. Use `ntpq -p` or `chronyc sources -v` to check synchronization status.

Q116



### How do you set up RAID0, RAID1, or RAID5 using mdadm?

For RAID0: `mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/sda1 /dev/sdb1`. For RAID1: use `--level=1` instead. For RAID5: use `--level=5` with three or more devices. After creation, format the array (e.g., with `ext4`), add an entry to `/etc/fstab` and update `mdadm.conf`.



Q117

**How do you analyze network traffic on a Linux host?**

Use tcpdump to capture packets for analysis; filter by protocol, port or host. Use Wireshark or tshark for graphical or command-line packet analysis. Tools like iftop and nethogs show real-time bandwidth usage per connection or process.

Q118

**What is a soft IRQ, and how does it differ from a hard IRQ?**

A hard IRQ is a hardware interrupt delivered directly to the CPU. A soft IRQ is a deferred interrupt handled in a software context to perform work that can be postponed, such as network packet processing. Soft IRQs run with interrupts enabled and can be scheduled on different CPUs.

Q119

**How do you use strace to debug a program?**

strace traces system calls made by a process. Run `strace ./program` to see all system calls. Use `-e trace=open,read,write` to filter calls. Attach to a running process with `strace -p PID`. This helps identify missing files, permission errors and unexpected behavior.

Q120

**What tools can you use to profile CPU usage at function level on Linux?**

Use `perf` or `perf record` with `perf report` to profile CPU hotspots. `gprof` works for instrumented binaries. eBPF-based tools like `bcc` and `bpftool` provide powerful tracing and profiling capabilities without modifying binaries.

Q121

**How do you build a highly secure SSH configuration?**

Disable password authentication (use key-based authentication), use SSH protocol 2 only, set a strong ciphers/MACs/KEX list, configure `AllowUsers` or `AllowGroups` to restrict login, enable two-factor authentication if possible, and run SSH on a non-standard port. Regularly review `/etc/ssh/sshd config`.